DRONATHON 3.0 · IEEE AESS/RAS ENSIT · 7–8 NOVEMBER 2026

Mission SKYFORGE

System design & execution plan

An autonomous coastal rescue drone in ROS 2 + Gazebo + PX4 that predicts where floating casualties and mines drift, triages casualties with a thermal camera, and keeps its mission alive through jamming, GNSS spoofing, forged commands and hardware failure.

< 50 ms reflex layer, p95	**≤ 100 ms** deterministic decision, p95	**0** collisions & zone entries	**< 1 KB** signed JSON messages

1	Mission and winning strategy
2	Idea verdicts
3	Team roles
4	System architecture: component map, mission sequence, pipeline, ROS 2 workspace
5	Simulation world and drone model
6	Perception: vision, thermal triage, drift prediction, datasets
7	Decision layer: state machine, triage score, RTH
8	Communication, security and resilience
9	Drone conception
10	Metrics and KPIs
11	Timeline, demo and risks

Team: Conception · P1 Sim & Control · P2 Perception · P3 Brain & Comms

1. Mission and winning strategy

We win by delivering one autonomous rescue drone that never fails during the demo, explains every decision live, and does three things few teams will: predicts where floating casualties and mines drift, triages casualties with a thermal camera, and survives jamming, spoofing and hardware failure.

Mission. A drone explores a simulated coastal zone in Gazebo, detects mines, weapons and marine debris, finds and ranks injured people, avoids hostile units and obstacles, drops simulated medical supplies on the right target, keeps working when the radio link dies, and returns home before the battery runs out. No human touches it.

Hard constraints from the spec.

- Reflex layer < 50 ms, decision ≤ 100 ms, total ≤ 150 ms at p95.
- No language-model call in the real-time loop. The orchestrator is LangGraph with deterministic nodes.
- No weapon or dangerous payload. Hostile units are zones to avoid, never targets.
- JSON messages < 1 KB with version and sequence number; ring buffer with declared capacity.
- Deliverables: Docker/ROS 2 code + 3D scene, technical docs, live demo, oral pitch, Raspberry Pi bonus.

How points are earned.

Criterion	Weight	What earns it for us
Autonomy	20%	Full mission from takeoff to landing with zero input, fault injection survived
Decision intelligence	20%	Thermal START triage, energy-gated scoring, drift intercept, live decision feed
Navigation	15%	A* around inflated zones, reflex avoidance of the moving boat, wind compensation
Communication and resilience	15%	Ring buffer + replay, HMAC anti-forgery, GPS spoofing detection, MAYDAY handover
Adaptability	15%	Instant goal_update reroute, drifting zones, wind-aware RTH threshold
Global efficiency	15%	Lawnmower coverage, energy per casualty, mission time
Bonus F12	extra	Comm module on Raspberry Pi with measured RAM, latency, resync time

Penalties are binary and expensive: collision, entering a danger zone, off-target delivery, impossible return. Every design choice below is biased toward never triggering one.

Three differentiators.

1. **Drift prediction.** A Kalman filter per floating object predicts where it moves with current and wind; the drone flies to the intercept point, and drifting mine zones grow and move.
2. **Thermal triage.** A simulated FLIR camera plus motion tracking sorts casualties into START categories: Immediate, Delayed, Minor, Expectant.
3. **Resilience layer.** Jamming, GPS spoofing, forged commands and motor failure are detected and handled live, ending in a MAYDAY handover to a replacement drone.

2. Idea verdicts

We keep nine of the team's ideas, reframe two so they score, and drop flares and Jev because they break the spec.

Idea	Verdict	How it lands in the project
Drift prediction (waves and current move what we detected)	**Keep, star feature**	Kalman filter per floating track, intercept planning, expanding-square re-search (section 6)
Pretrained vision models	Keep	YOLO11n pretrained on COCO, fine-tuned on simulator + aerial data
FLIR thermal: injured before unresponsive	Keep, reworded	START categories from heat + motion. Thermal cannot reliably prove death (bodies cool over hours), so the lowest class is called Expectant / unresponsive
RTH with estimated return energy	Keep	Wind-aware threshold every tick; predicted vs actual energy shown as a metric
Carbon fibre / fibreglass, anti-corrosion	Keep (conception)	Carbon frame, fibreglass radome for antennas (carbon blocks RF and GNSS), marine coating, IP67
Radar-absorbent coating	One line only	Composite airframe already has a small radar signature; mention it, don't build on it
Evasive manoeuvres	Reframe	Threat-aware planning: avoid each hostile unit's detection radius and line of sight, fly low, replan instantly
Flares against heat seekers	**Drop**	Pyrotechnics are a dangerous payload (spec section 2) and contradict the humanitarian rule. Replace with humanitarian marking + identification beacon: medical units are protected when identifiable
Warship dataset for conflict zones	Keep, cheap	Vessels join the hostile_unit class; each detection creates a growing exclusion zone
Jamming / attacks that break the drone	Keep, solved	Resilience layer in section 8: GNSS denial, spoofing, radio jamming, forged commands, hardware faults
Drone tells the station to send a replacement	**Keep, star feature**	MAYDAY message with mission snapshot; drone 2 resumes the mission
Interconnected swarm	Future work	One slide; MAYDAY handover already shows the idea
Jev (TypeSafe AI) in the loop	**Drop**	70–500 ms claimed latency + network round trip breaks the 100 ms budget, needs internet (fails F10), and is an AI model in the real-time loop

3. Team roles

One person owns the drone conception and the story; three engineers each own a vertical slice of the software with clear interfaces, so all four work in parallel from hour one.

Role	Owns	Delivers to others	Second job after core is done
Conception (hardware + story)	CAD, materials, sensors, mass and power budget, payload drop, drone visual in Gazebo	Power model numbers to Brain (for RTH), sensor specs to Perception (FOV, resolution)	Slides, architecture diagrams, documentation layout
P1 Sim & Control	Docker, PX4 SITL, Gazebo coastal world, wind, obstacles, moving boat, thermal models, reflex node	Running sim, odometry and camera topics, `/fmu/in` setpoint interface	Foxglove dashboard, fault injector, truth logger
P2 Perception	Both vision nodes, thermal node, tracking, 2D→3D, drift Kalman filter, dataset generation and training	`/tracks` and `/zones` with uncertainty	Vision metrics vs ground truth
P3 Brain & Comms	LangGraph orchestrator, planner (coverage, A*, triage, RTH), comm node, ring buffer, security, MAYDAY	`/plan/waypoints`, `/mission/state`, JSON link to ground station	Raspberry Pi bonus, mission_logs export

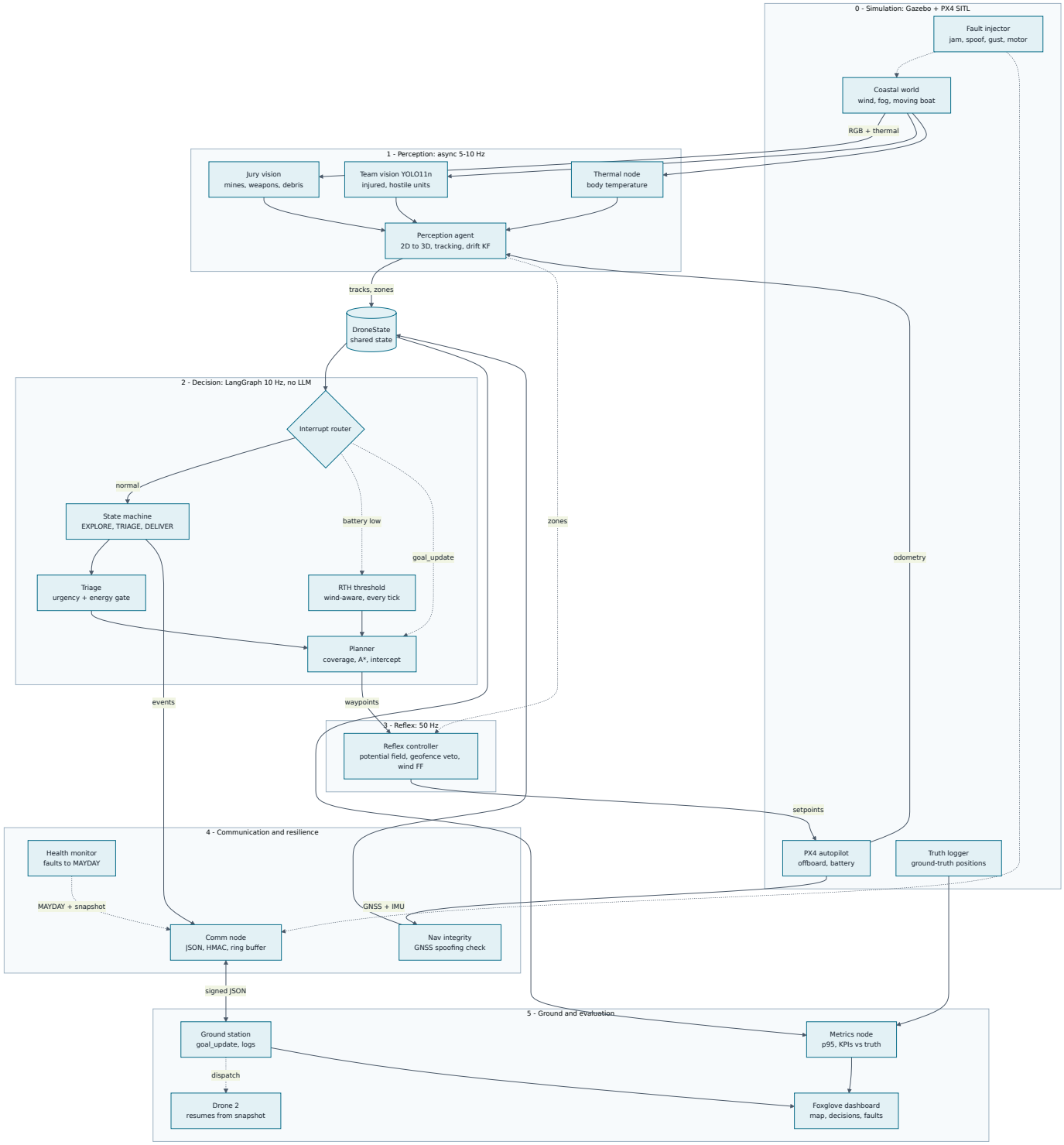
Rules that keep four people unblocked.

1. Interfaces are frozen in hour 1: `skyforge_interfaces` messages and the JSON schema (sections 4 and 8). Change them only with all four in the room.
2. Every node runs alone with a fake publisher. P2 tests on recorded camera bags, P3 tests with a fake `/tracks` publisher, P1 tests reflex with a fixed waypoint.
3. One Git repo, one `main` that always launches. Work on branches, merge only what launches.
4. Parameters live in `params.yaml`, never hard-coded, so tuning during stress tests needs no rebuild.

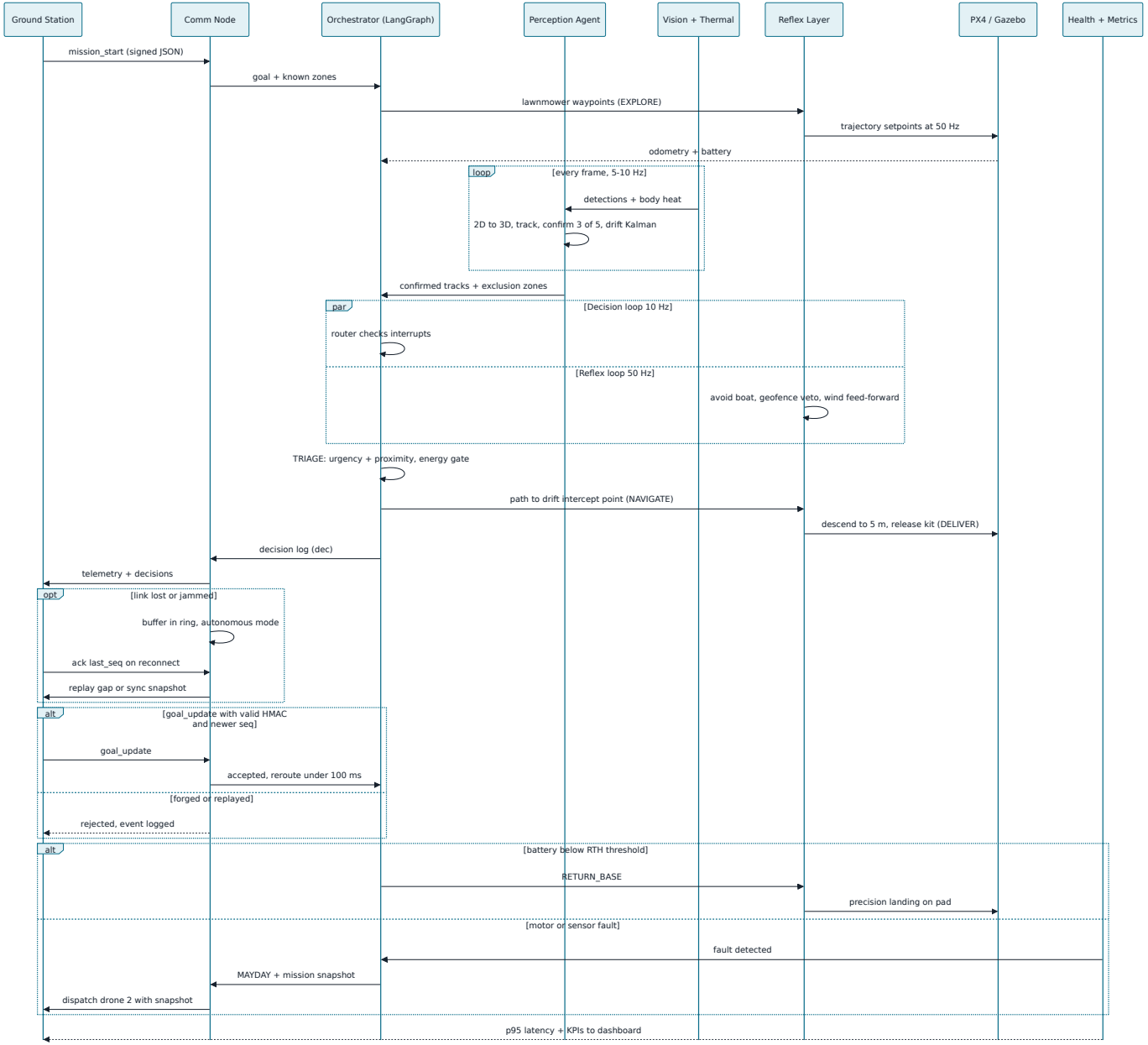
4. System architecture

The system is split into a fast reflex layer (50 Hz) that keeps the drone safe and a deterministic decision layer (10 Hz) that decides what to do; perception runs asynchronously beside them and never blocks the loop.

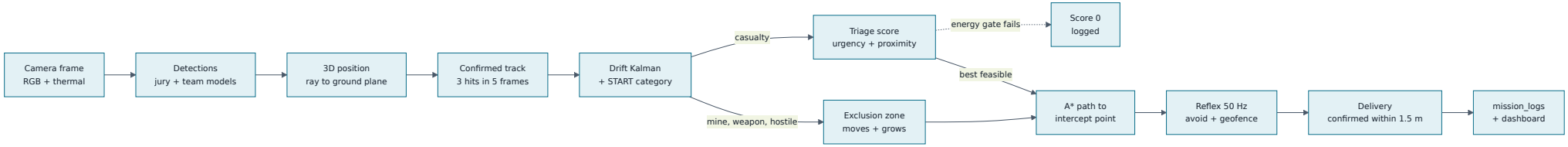
4.1 Component map



4.2 Mission sequence



4.3 Pipeline from pixel to delivery



4.4 Stack and workspace

Stack. ROS 2 Humble · Gazebo Harmonic · PX4 SITL (x500 with downward camera) · Micro XRCE-DDS agent on UDP 8888 · `ros_gz_bridge` · Python 3.10 (rclpy, numpy, ultralytics, onnxruntime, langgraph, filterpy) · Foxglove Studio · Docker Compose. Go/no-go: if PX4 is not flying in Docker by hour 2, fall back to a velocity-controlled Gazebo model and keep everything else.

ROS 2 workspace `skyforge_ws/src/`.

Package	Nodes / content	Owner
skyforge_interfaces	Msgs: Detection, DetectionArray, Track, TrackArray, Zone, ZoneArray, Casualty, MissionState, MissionEvent, Metric	P3 (hour 1)
skyforge_bringup	`sim.launch.py`, `mission.launch.py`, `params.yaml`, `worlds/coastal.sdf`, `models/`	P1
skyforge_reflex	`reflex_node` 50 Hz: potential-field avoidance, geofence veto, wind feed-forward, critical-battery RTH, PX4 offboard bridge	P1
skyforge_perception	`jury_vision_node`, `team_vision_node`, `thermal_node`, `perception_agent_node` (fusion, 2D→3D, tracking, drift, zones)	P2
skyforge_brain	`orchestrator_node` (LangGraph StateGraph), lib: `coverage.py`, `astar.py`, `triage.py`, `rth.py`, `intercept.py`	P3
skyforge_comm	`comm_node` (JSON codec, HMAC, ring buffer, link simulator), `ground_station_node`	P3
skyforge_monitor	`health_node` (fault detection, MAYDAY), `metrics_node` (latency p50/p95/p99, KPIs)	P3 / P1
skyforge_sim_tools	`dataset_generator`, `moving_obstacle`, `fault_injector`, `truth_logger`	P1 / P2

Topics.

Topic	Type	Rate	From → To
`/cam/rgb`, `/cam/thermal`	sensor_msgs/Image	10–15 Hz	Gazebo bridge → vision nodes
`/cam/rgb/camera_info`	sensor_msgs/CameraInfo	1 Hz	bridge → perception_agent
`/fmu/out/vehicle_odometry`	px4_msgs/VehicleOdometry	100 Hz	PX4 → all (BEST_EFFORT QoS)
`/fmu/out/battery_status`	px4_msgs/BatteryStatus	10 Hz	PX4 → brain, health
`/vision/jury`, `/vision/team`	DetectionArray	5–10 Hz	vision nodes → perception_agent
`/perception/tracks`	TrackArray	10 Hz	perception_agent → orchestrator
`/perception/zones`	ZoneArray	10 Hz	perception_agent → orchestrator, reflex
`/plan/waypoints`	nav_msgs/Path	on change	orchestrator → reflex
`/mission/state`	MissionState	10 Hz	orchestrator → comm, dashboard
`/mission/events`	MissionEvent	on event	all → comm, metrics (feeds mission_logs)
`/station/goal_update`	std_msgs/String (JSON)	on event	comm → orchestrator
`/fmu/in/trajectory_setpoint`	px4_msgs/TrajectorySetpoint	50 Hz	reflex → PX4
`/metrics`	Metric	1 Hz	metrics_node → dashboard

PX4 integration rules. PX4 uses NED frames, ROS uses ENU: one conversion module, used everywhere. Subscribe to `/fmu/out/*` with BEST_EFFORT QoS. Stream `offboard_control_mode` + `trajectory_setpoint` at ≥ 10 Hz before switching to offboard and arming. Check exact topic names with `ros2 topic list` (some PX4 versions add a `_v1` suffix).

Docker Compose services. `sim` (PX4 + Gazebo + XRCE agent), `brain` (orchestrator + reflex + monitor), `vision` (vision + perception agent), `comm` (comm node, also runs on the Raspberry Pi), `ground_station`, `foxglove_bridge`. All share `network_mode: host` and one `ROS_DOMAIN_ID`.

5. Simulation world and drone model

The world `coastal.sdf` contains every condition the jury will test, plus a ground-truth logger so every metric is measured against reality, not guessed. If the jury provides its own scenario, we load it and keep only our additions (thermal plugins, truth logger, fault injector).

Element	Implementation
Coastline, sand, water, rocks	Heightmap or flat planes with textures; water plane at z = 0
Wind	`gz::sim::systems::WindEffects` in the world, or PX4 `windy` world; gusts changed live by fault_injector
Visibility	Scene fog parameters, varied per run
Fixed obstacles	Rocks, poles, a lighthouse, a cliff edge
Moving obstacle	A boat model moved by `moving_obstacle` through Gazebo's `set_pose` service on a looping path
Casualties	Person models with `gz::sim::systems::Thermal` : ~310 K warm, ~295 K cold; lying, sitting, standing poses; some in water
Drift	`moving_obstacle` also drifts floating casualties and mines with a set current + leeway, ground truth logged
Mines, weapons, debris	Jury-provided models; mines on beach and floating
Hostile units	Armed figures, vehicles, a patrol vessel; each with a declared detection radius
Radio shadow zones	Polygons in `params.yaml` , enforced by the comm link simulator
Base	Landing pad with an ArUco marker for precision landing

Drone sensors in sim. Downward RGB camera (from PX4 `x500_mono_cam_down`), a `thermal_camera` sensor on the same mount, PX4's simulated IMU, GNSS, barometer and battery. The conception's mesh replaces the x500 visual; physics stays the x500's so control tuning is free.

Truth logger. Records true positions of every object and the drone at 10 Hz into `truth.csv` . Every accuracy metric in section 10 is computed against it.

6. Perception

Perception turns two detectors and a thermal camera into confirmed 3D tracks with uncertainty, triage categories and moving exclusion zones; it runs at 5–10 Hz in its own process and the decision loop always reads the latest result without waiting.

Pipeline.

1. `jury_vision_node`: wraps the jury model (debris, mines, weapons) and publishes the common `DetectionArray` (class, confidence, bbox).
2. `team_vision_node` (F5): YOLO11n, classes `injured`, `hostile_unit` (people, vehicles, vessels), exported to ONNX for CPU. Low threshold (0.15–0.25) on safety classes for recall.
3. `thermal_node`: for each person box, reads the matching thermal region and outputs mean body temperature and contrast to background.
4. `perception_agent_node`: projection to 3D, association, confirmation, drift filter, triage category, exclusion zones.

2D → 3D. Take the bbox bottom-centre pixel, build a ray with the camera intrinsics, rotate it to world with the drone attitude and camera mount, intersect with the plane z = 0 (water or beach). Position uncertainty grows with altitude and attitude error; it is stored in the track. Target: < 2 m error vs truth.

Tracking and false-detection filter. Associate detections to tracks within 3 m (nearest neighbour or ByteTrack in image space). A track is confirmed after 3 hits in 5 frames at a consistent position. Unconfirmed tracks never trigger decisions; they only slow the drone to look again.

Thermal START triage.

Category	Rule (thermal + motion + context)	Urgency
Immediate	Warm, immobile for > 10 s, lying; or in water without flotation	1.0
Delayed	Warm, lying or sitting, moving	0.7
Minor	Warm, standing or waving	0.3
Expectant / unresponsive	Near ambient temperature, immobile	0.1

Motion = track centroid displacement and box-shape change over a 10–30 s window. Proximity to a mine raises urgency by one step.

Drift prediction. Each floating track runs a constant-velocity Kalman filter with state [x, y, vx, vy].

$$\mathbf{v}_{drift} = \mathbf{v}_{current} + k_{leeway}\,\mathbf{v}_{wind}, \qquad \mathbf{p}(t+\Delta t) = \mathbf{p}(t) + \mathbf{v}_{drift}\,\Delta t$$

- Prior drift velocity = sea current + leeway (a few percent of wind speed, downwind; US Coast Guard leeway tables).
- Seen → filter update; not seen → predict only, covariance grows, drawn as an ellipse on the dashboard.
- Planner flies to the intercept point p(t_arrival), solved by 3 fixed-point iterations.
- Not found at the prediction → expanding-square search centred on it, size from the covariance.
- Floating mines: their exclusion circle moves with the filter and its radius = safety distance + 2σ.

Exclusion zones. Radius = class safety distance (mine 15 m, weapon 10 m, hostile unit = its declared detection radius) + localisation uncertainty + wind margin. Zones are hard constraints for both the planner and the reflex geofence.

Datasets and training.

Source	Use
Kaggle gun-detection (1,310 images, human + gun)	Weapons; remap the 14 unused classes; ground-level, so fine-tune on sim crops
Roboflow mine detection	Mines; check classes and licence first
Zenodo TUN-MarineLitter (Mahdia coast, ENSIT, 7 debris classes, polygon labels)	Debris; convert polygons to boxes
Gazebo bounding-box camera, 1,500–3,000 frames	Injured, hostile units, sim mines and weapons; randomised altitude 10–40 m, pitch, light, fog, poses
C2A (pose-labelled disaster people)	Lying / sitting people from the air
SeaDronesSee (swimmer, floater, boat)	People in water for the coastal mission
Military UAV dataset (tank, soldier, people)	Hostile vs civilian

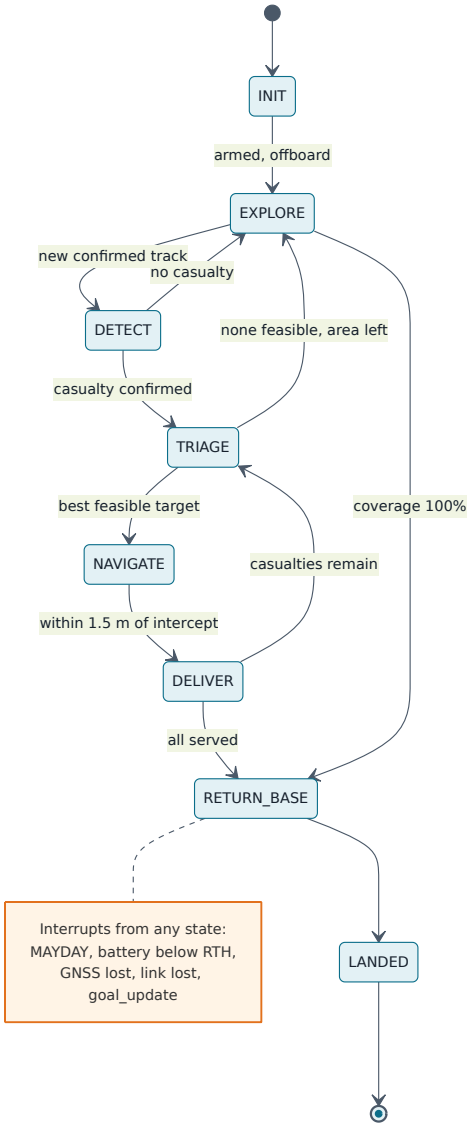
Train on a free GPU (Kaggle or Colab), 640 px, ~50 epochs, split simulator frames by scene to avoid leakage. Export ONNX, benchmark p95 on the demo laptop under full load.

7. Decision layer

The orchestrator is a LangGraph StateGraph with pure-Python nodes ticking at 10 Hz (target p95 < 20 ms); every decision is a formula whose inputs and output are logged to mission_logs.

DroneState (shared TypedDict). position, velocity, attitude, battery_wh, battery_pct, link_up, link_quality, gnss_ok, nav_source, wind_vec, visibility, threats (zones), casualties (tracks + category + drift), served_ids, current_goal, waypoints, phase, interrupts, rth_threshold_wh, logs.

7.1 State machine



A router node runs first every tick and checks interrupts in this priority order:

1. MAYDAY (health fault) → emergency handover + controlled landing away from casualties.
2. Battery below RTH threshold → RETURN_BASE.
3. GNSS lost or spoofed → switch nav source, inflate margins, recompute RTH.
4. Link lost or jammed → autonomous flag, mission continues, messages buffered.
5. goal_update received → reroute immediately (target < 100 ms reaction).
6. New threat zone → replan the current path.

Node functions don't depend on LangGraph, so a plain dict-dispatch FSM can replace it in 10 minutes if its overhead ever shows in p95.

7.2 Planning, triage, delivery, RTH

Exploration. Boustrophedon (lawnmower) sweep; lane spacing = camera ground footprint × 0.8 overlap at cruise altitude 25 m. Lanes clipped by exclusion zones. Coverage tracked on a 2 m grid.

Path planning. A* on a 1 m 2D grid at cruise altitude; obstacles and exclusion zones inflated by drone radius + safety margin (+ extra with wind speed and low visibility). Path smoothed, then replanned whenever a zone, goal or casualty changes. Threat-aware cost: cells inside a hostile unit's line of sight cost 10× more.

Triage score (F6).

$$S_i = \begin{cases} 0.7\,U_i\,c_i + 0.3\,P_i & \text{if } E_{d\to i} + E_{i\to base}(w) + E_{res} \le E_{avail} \\ 0 & \text{otherwise} \end{cases} \qquad P_i = 1 - \frac{d_i}{d_{max}}$$

U = urgency from section 6, c = detection confidence, P = proximity, w = wind. Ties broken by time since detection. Each decision logs all candidates, their components and why the others lost.

Delivery (F7). Fly to the drift-predicted intercept point, descend to 5 m, confirm the casualty track is within 1.5 m of the drop point, release. Not confirmed → abort, re-acquire, retry. Off-target release is impossible by design.

Return to home (F11). Recomputed every tick:

$$v_g = v_{air} - \mathbf{w}\cdot\hat{\mathbf{u}}_{home}, \qquad E_{RTH} = P(v_{air}, w)\,\frac{d_{home}}{\max(v_g, v_{min})} + E_{land} + E_{res}$$

RETURN_BASE is forced when E_avail ≤ 1.3 × E_RTH. P(v, w) comes from the conception's power budget, calibrated by a 60 s hover and a 60 s cruise in the sim. E_res = 15% of capacity. With GNSS lost, the factor rises to 1.5.

Reflex layer (50 Hz, target < 5 ms per tick). Follows the waypoint path, adds a potential-field repulsion from obstacles (moving boat predicted at constant velocity), projects the commanded velocity off any exclusion-zone boundary (hard veto), adds wind feed-forward from the low-passed difference between commanded and achieved velocity, and triggers RTH alone if battery hits the critical floor.

8. Communication, security and resilience

The link is designed to lose nothing: every message is small, numbered, authenticated and buffered, and the drone keeps flying its mission when the link, GNSS or a motor fails.

Message format (JSON, < 1 KB, asserted in code).

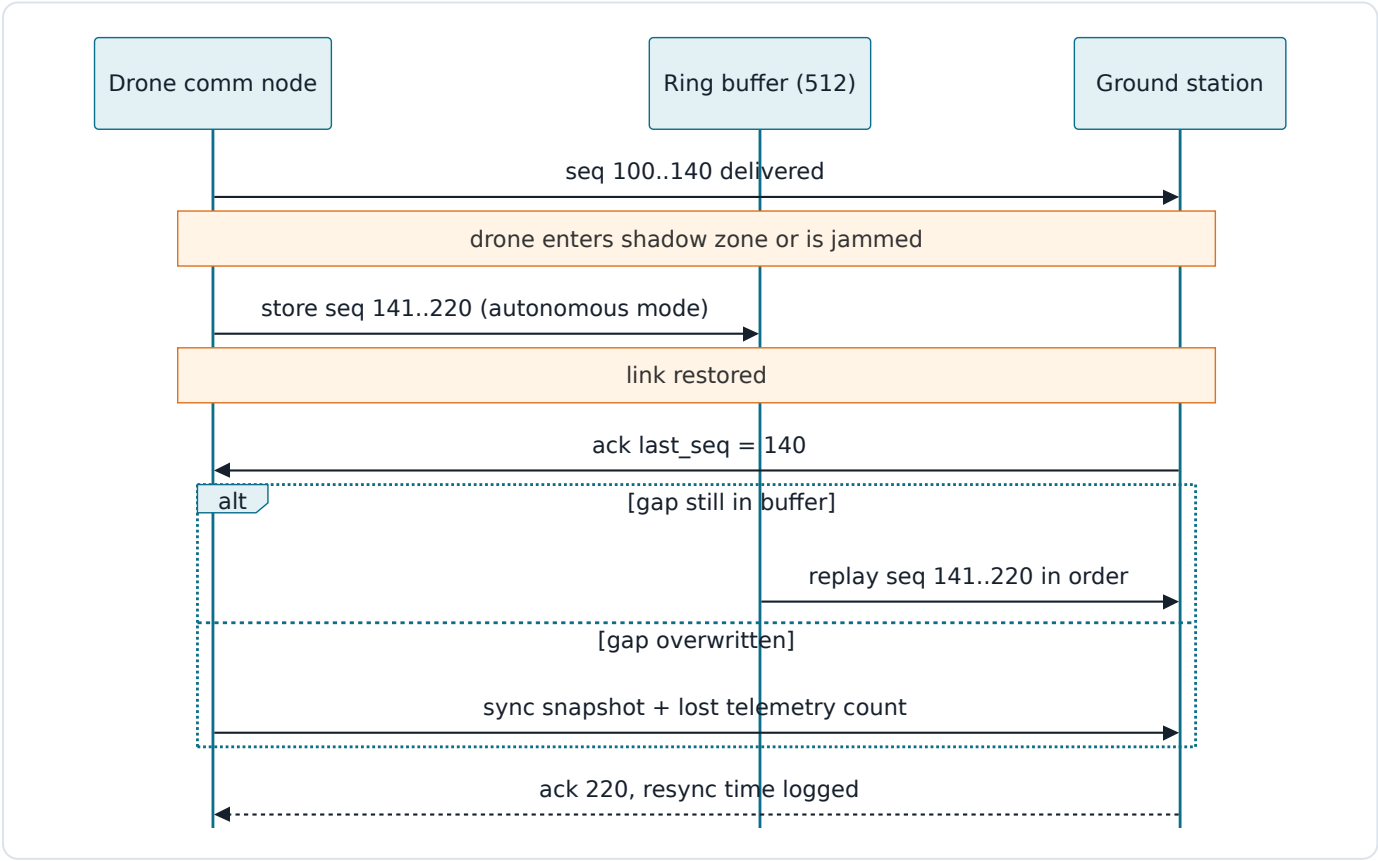
```
{"v":1,"seq":1832,"ts":1762560000.12,"t":"det","id":"C3","c":"injured","u":"IMM",
 "p":[41.2,-12.8,0],"vd":[0.31,0.12],"cf":0.87,"h":"9f2a1c7e4b0d3a11"}
```

Type `t`	Content	Priority in buffer
`tel`	Position, battery, phase, link, nav source	Low (overwritten first)
`det`	Confirmed detection: class, category, position, drift	High
`dec`	Triage / delivery / RTH decision summary	High
`goal`	goal_update from station	Critical
`ack`	last_seq_received	Critical
`sync`	State snapshot after long outage	Critical
`may`	MAYDAY + mission snapshot	Critical

Ring buffer. Declared capacity 512 messages (< 512 KB worst case). While the link is down, every outgoing message is stored. On reconnect the station sends its last received seq; the drone replays the gap in order. If the gap was overwritten, it sends a `sync` snapshot plus the count of lost telemetry. High-priority messages are never overwritten by telemetry.

Link simulator. Link is up unless the drone is in a shadow polygon, beyond max range, or jammed. Jamming is modelled as packet loss + a raised noise floor, controlled by the fault injector.

8.1 Link-loss and resync flow



Threats and responses.

Threat	Detection	Response
Radio jamming	Packet loss rises, noise floor rises	Autonomous mode, buffer everything, steer away from the estimated jammer direction to regain link
GNSS jamming	Fix lost, satellite count drops	Dead reckoning from IMU + optical flow; uncertainty grows, RTH factor 1.3 → 1.5
GNSS spoofing	GNSS position jumps inconsistent with IMU/odometry estimate (innovation test)	Reject GNSS, flag on dashboard, continue on odometry
Forged or replayed command	HMAC-SHA256 tag (truncated to 8 bytes, shared key) + seq must be newer	Reject, log `forged_rejected` event
Hardware damage (motor, sensor, overheating)	health_node: motor output saturation, sensor timeout, battery voltage sag	MAYDAY with snapshot, controlled landing away from casualties
Directed energy / EMP	Out of simulation scope	Conception: shielded avionics, redundant IMU, fibreglass radome

MAYDAY handover. The failing drone sends `may` with: remaining casualties (id, category, drift state), exclusion zones, coverage grid hash, last position. The station starts drone 2 from base with that snapshot as its initial DroneState; it skips explored areas and goes straight to TRIAGE. Stretch goal: spawn drone 2 live in the demo.

Raspberry Pi bonus (F12). Only `comm_node` runs on the Pi as a UDP/WebSocket bridge between brain and ground station. Measured: RAM (psutil RSS), CPU %, per-message processing latency, round-trip time, resync time after a forced 30 s outage.

9. Drone conception

The physical design is a marine-hardened quadrotor whose mass and power numbers feed the software's energy model directly, so the hardware and the code tell one story.

Deliverables for the conception owner.

- ☐ CAD (Onshape or Fusion 360): frame, arms, payload box with servo latch, sensor gimbal, sealed battery bay, fibreglass radome
- ☐ Mass and power budget spreadsheet: thrust-to-weight ≥ 2, hover and cruise power, battery Wh, endurance; numbers handed to P3 for P(v, w)
- ☐ Materials and protection sheet
- ☐ Simplified mesh exported and placed on the x500 in Gazebo
- ☐ Exploded view + sensor layout image for the slides

Area	Choice	Why
Frame	Carbon fibre arms and plates	Stiff, light, corrosion-resistant
Antenna mast / radome	Fibreglass	Carbon fibre blocks RF and GNSS; fibreglass is RF-transparent
Corrosion	Marine conformal coating on electronics, stainless fasteners, sealed motors, IP67 target	Salt spray on a coastal mission
Payload	Medical kit pod, servo latch, drop from ~5 m with a small drag chute	No dangerous payload; accurate low-altitude release

Area	Choice	Why
Perception	Downward RGB + radiometric thermal camera (e.g. FLIR Boson / Lepton class) on a 2-axis gimbal	Thermal triage, night and fog capability
Navigation	Dual IMU, GNSS, optical-flow sensor, downward rangefinder	Dead reckoning when GNSS is jammed
Comms	Main radio + backup low-rate radio, companion computer (Raspberry Pi)	Resilience; matches the F12 bonus
Identification	Humanitarian markings + identification beacon and strobe	Protected status depends on being identifiable
Signature	Composite airframe, low profile	Small radar signature as a side benefit, not a design goal
Safety	Prop guards, parachute option, failsafe landing	Operating near casualties

Present the payload, the thermal camera and the fibreglass radome as the three hero details; judges remember specifics.

10. Metrics and KPIs

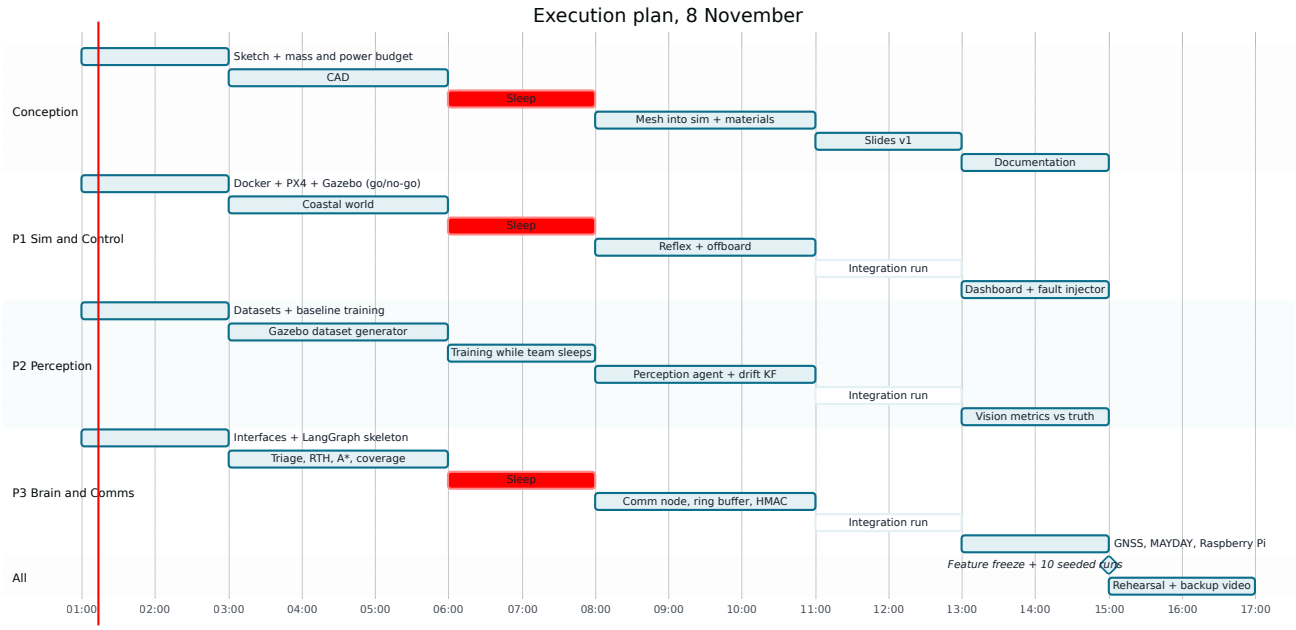
Every metric is measured automatically, shown live on the Foxglove dashboard, and computed against the truth logger over 10 seeded runs for the final slide.

Category	Metric	Target	How measured
Real-time	Reflex / decision / total latency p50, p95, p99	< 50 / ≤ 100 / ≤ 150 ms at p95	`perf_counter_ns` around every node, rolling buffer, numpy percentiles
Vision	Precision, recall, mAP@50, mAP@50-95 per class	Recall ≥ 0.95 on mines and weapons	Ultralytics val on the 10% test split + sim test scenes
Vision	Inference time p95, FPS, detection age	p95 < 100 ms (async)	Timestamps in DetectionArray
Localisation	3D position error of confirmed tracks	< 2 m	Track vs truth.csv
Drift	Prediction error vs time unseen	Plot; < 5 m after 30 s	Filter prediction vs truth
Exploration	Coverage % over time	100% of reachable area	2 m coverage grid
Safety	Collisions, danger-zone entries, minimum clearance	0, 0, > safety radius	Truth distances every tick
Triage	Rescue order vs oracle order (Kendall τ)	τ = 1	Oracle ranks with true categories and positions
Triage	Casualties served / total, time to first delivery	All feasible served	mission_logs
Delivery	Drop point error vs true casualty position	< 1.5 m	Drop event vs truth
RTH	Predicted vs actual return energy; battery at landing	Error < 10%; ≥ reserve	Energy log at trigger and at landing
Comms	Max and mean message size	< 1 KB	Size logged per message
Comms	Packet loss, outage durations, messages replayed, resync time, buffer occupancy	0 decision logs lost	comm_node counters
Adaptability	goal_update reaction time	< 100 ms	Receive timestamp → new path published
Security	Forged and replayed messages rejected; spoofing detection time	100%; < 2 s	Fault injector counts vs rejections
Bonus F12	Pi RAM, CPU, per-message latency, resync time	Reported	psutil + timestamps
Efficiency	Mission time, distance, energy per casualty served	Reported	mission_logs

Dashboard panels. 3D/2D map (trajectory, zones, casualties coloured by category, drift ellipses), FSM state, battery vs RTH threshold gauge, link and nav-source status, latency histogram with p95 line, live decision feed (one templated sentence per decision), fault-injection buttons.

11. Timeline, demo and risks

Feature freeze is at 15:00 on 8 November; everything after that is testing, metrics, slides and rehearsal. Shift all times if the final deadline is different.



Demo script (5 minutes, zero human input on the drone).

1. Takeoff; dashboard shows INIT → EXPLORE and the lawnmower path.
2. Mine detected → exclusion zone appears, path replans around it.
3. Casualties detected; thermal categories appear; decision feed explains the triage order.
4. Fault: jam the radio → LINK LOST, autonomous mode, buffer fills; link back → replay and resync time shown.
5. Fault: spoof GNSS → rejected, nav source switches to odometry.
6. Fault: send a forged goal_update → rejected; send a real one → reroute in < 100 ms.
7. Fault: wind gust → RTH threshold line jumps on the gauge.
8. A floating casualty drifts; the drone flies to the predicted intercept and delivers within 1.5 m.
9. Fault: motor failure → MAYDAY, drone 2 takes over from the snapshot (stretch), drone 1 lands safely.
10. Metrics slide: p95 latency, 0 collisions, 0 zone entries, delivery error, recall per class, Pi measurements.

Pitch structure. Problem (30 s) → architecture (60 s) → live demo (2 min 30) → metrics (40 s) → conception and future work: swarm relay, onboard thermal vitals (20 s).

Risks and fallbacks.

Risk	Fallback
PX4 SITL not flying in Docker by 03:00	Velocity-controlled Gazebo model, own wind compensation
Jury scenario or simulator differs	Load theirs; keep our nodes, thermal plugin and truth logger
Detector too weak on sim imagery	Fine-tune on 300 sim screenshots of jury models; lower threshold + temporal confirmation
LangGraph overhead in p95	Swap to dict-dispatch FSM (same node functions)

Risk	Fallback
Laptop CPU overloaded during demo	Vision at 5 Hz, headless Gazebo render for sensors, close everything else
Live demo crashes	Backup video recorded after freeze; seeded run that is known to pass
Drone 2 spawn unstable	Show MAYDAY + snapshot only; drone 2 becomes future work